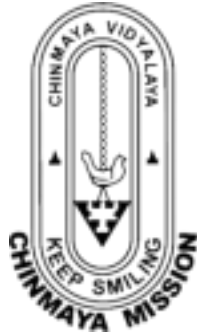




CHINMAYA VIDYALAYA



COMPUTER SCIENCE

--TOPIC--

FLIGHT RESERVATION AND MANAGEMENT SYSTEM

Name- SHRIYA SHANKAR

Class- 12 Science

Batch- 2025-26

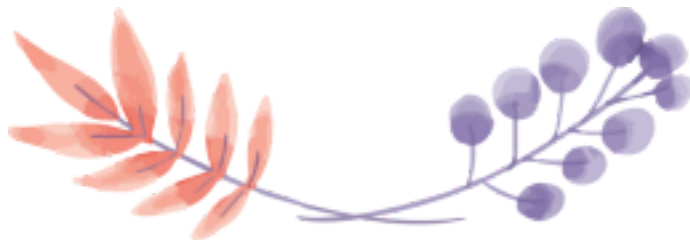


TABLE OF CONTENTS....

- Acknowledgement
- Certificate
- Introduction To 'PYTHON'
- About The Function/Modules/Libraries
Used. • Software And Hardware Requirement
- SOURCE CODE
- Output Screen
- Advantages And Disadvantages
- Bibliography



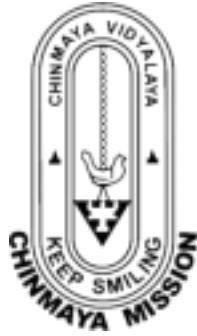
ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all the people who have helped me in completion of this project on time. I would like to thank our Computer Science teacher Mrs.P.Sulakshana for guiding and motivating us throughout the project.

I am also grateful to my friends, teammates and family for their support throughout the completion of this project.



CERTIFICATE



This is to certify that SHRIYA SHANKAR,
student of class 12 (Science Stream) has
successfully completed her project on “FLIGHT
RESERVATION AND MANAGEMENT SYSTEM”
under my guidance, and as prescribed by CBSE
for the academic year 2025-26.

Internal Examiner

External Examiner School Stamp



INTRODUCTION

Python is a popular, high-level, easy-to-read, general purpose programming language known for its simple syntax, making it great for beginners and experts alike; it's used for web development, data science, AI, automation, and more, with strong library support and versatility. Created by Guido van Rossum in the 1990s, it emphasizes code readability and supports multiple programming paradigms, allowing developers to write clearer, more logical code for projects of all size.



○ **Key Characteristics:-**

- **Easy to Learn:** Simple, readable syntax reduces the cost of program maintenance and speeds up development.
- **High-Level:** Abstracts away complex memory management and system details.
- **Interpreted:** Code is executed line-by-line, making the edit test-debug cycle very fast.
- **Object-Oriented:** Supports object-oriented programming (OOP) principles.
- **Free & Open-Source:** Available for free with extensive standard libraries and community support.

○ **Common Uses: -**

- **Web Development:** Building web applications and backend services.
- **Data Science & Machine Learning:** Analysis, visualization, and building AI models.
- **Automation:** Scripting repetitive tasks and automating workflows.
- **Software Development:** Prototyping, testing, and building software



○ About the project

This project is an interface of python with MySQL. For the source code to run both the paths (SQL and python) need to be added in the system. Python IDLE and MySQL need to be installed in the device. We need to install the necessary pip packages. There is use of many libraries and modules, we can check the connectivity by importing 'mysql.connector' as 'sqltor' and connect by using 'mysql.connect' and check the connectivity using 'is_connected' function. Python is case sensitive so everything should be typed in the same case as it is done in MySQL.

- This project's topic Flight Reservation and Management System serve a user-friendly platform for passengers and an efficient management for the staff.
- This system automates tasks such as searching for available flights, booking the ticket, reservation and updating the flight schedules.
- It eliminates the need for manual intervention reducing errors and enhancing user satisfaction. The system ensures real time problems and supports essential functions like seat allocation, ticket booking.
- By integrating python as the main programming language and SQL as the database management system provides a robust and a scalable solution for the real-world airline problems.
- The primary goal of this project is to simplify flight management by offering a seamless user interface, a reliable backend, and secure the data handling.
- We have used the MySQL connector in order to connect SQL with python and execute the queries on the MySQL databases.



○ About SQL: -

SQL is the standard language for communicating with relational databases, allowing users to interact with and manipulate data stored in tables of rows and columns. It was standardized by the American National Standards Institute (ANSI) in 1986 and remains widely used today

Commands: SQL commands are generally grouped into categories such as:

- **DDL (Data Definition Language):** for defining and modifying database structures (e.g., CREATE, DROP).
- **DML (Data Manipulation Language):** for manipulating data within tables (e.g., INSERT, UPDATE, DELETE).
- **DQL (Data Query Language):** for retrieving data (e.g., SELECT).



Function/Modules/Libraries Used

- **import :-**

Import is one of the keywords in python. It allows you to import a function from a module instead of importing everything. When python encounters this command python starts processing the module. The () and variables of that particular module are now available for that program.

- **append() :-**

The append function in python adds a single element at the end to an existing list. It does not return a new list of items but will modify the original list by adding an item in the end. After executing the method append the list size will increase by one.

- **Select :-**

Select is a DML command used to query the database. A query is a command which is given to produce the desired information from the database table.

- **input() :-**

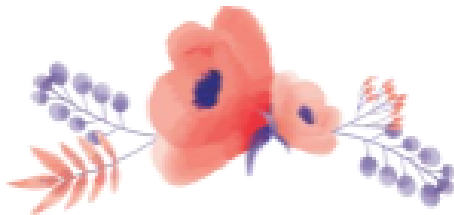
This function always returns a value of the type string. Python offers two functions int() and float() to be used to convert values received through input to the types int and float.



Software And Hardware

Requirement

- **Hardware:** -
 - Computer
 - Keyboard
 - CPU
 - Mouse
 - Storage
- **Software:** -
 - Microsoft Word
 - Google
 - Chrome
 - Python IDLE Version 3.11
 - MySQL Command Line Client
 - Libraries



SOURCE CODE

```
create database hotel;  
use hotel
```

```
create table pdata  
(custname varchar(30),  
addr varchar(100),  
jrdate varchar(10),  
source varchar(10),  
destination varchar(10));
```

```
create table classtype  
(sno varchar(5),  
classtype varchar(15),  
price int(10));
```

```
insert into classtype values ('1','firstclass', 6000);  
insert into classtype values ('2','businessclass', 4000);  
insert into classtype values ('3','economyclass', 2000);  
select * from classtype;
```

```
create table food  
(sno int,  
itemname varchar(10),  
rate float);
```

```
insert into food values(1, 'tea', 10);  
insert into food values(2, 'coffee', 10);  
insert into food values(3, 'coldrink', 20);  
insert into food values(4, 'samosa', 10);  
insert into food values(5, 'sandwich', 50);  
insert into food values(6, 'noodles', 50);  
insert into food values(7, 'pasta', 50);  
select * from food;
```

```
create table luggage  
(sno int,  
weight varchar(10),  
rate float);
```

```
insert into luggage values(1, '20kg', 1000);  
insert into luggage values (2, '25kg',1500);  
insert into luggage values (3, '30kg', 2000);  
insert into luggage values (4, '150kg', 3000);
```

```
Select * from luggage;
```

```
#PYTHON
```

```
import mysql.connector  
from mysql.connector import Error
```

```
# Database connection
```

```
try:
```

```
    con = mysql.connector.connect(  
        host="localhost",  
        database="hotel",  
        user="root",  
        password=""  
    )
```

```
    mycursor = con.cursor()
```

```
except Error as e:
```

```
    print("Error connecting to database:", e)  
    exit()
```

```
# Function to register a customer
```

```
def registercust():
```

```
    customer_data = []
```

```
    name = input('Enter name: ')
```

```
    address = input('Enter locality: ')
```

```
    city = input('Enter city: ')
```

```
    state = input('Enter state: ')
```

```
    pin = input('Enter PIN code: ')
```

```
    jr_date = input('Enter date of journey  
(YYYY-MM-DD): ') source = input('Enter  
source: ')
```

```
    destination = input('Enter destination: ')
```

```
    fulladd = address+', '+city+', '+state+', '+pin
```

```
    customer_data.extend([name, fulladd, jr_date, source, destination])
```

```
# Insert into SQL
```

```
sql = """INSERT INTO pdata  
    (custname, addr, jrdate, source, destination)  
    VALUES (%s,%s,%s,%s,%s)"""
```

```
try:
```

```
    mycursor.execute(sql, customer_data)
```

```
    con.commit()
```

```
    print("Customer registered successfully.\n")
```

```
except Error as e:
```

```
    print("Error inserting data:", e)
```

```
# Write to bill file
with open("Bill.txt", 'a') as f:
    f.write("----- Customer Details -----\\n")
    f.write(f"Name: {name}\\nState: {state}\\nCity: {city}\\nAddress:
{address}\\n") f.write(f"PIN code: {pin}\\nJourney date:
{j_r_date}\\nSource: {source}\\nDestination: {destination}\\n\\n")
```

```
# Function to calculate ticket price
def ticketprice():
    print('We have the following rooms for you:')
    print("1. First class -> Rs 7000 PN")
    print("2. Business class -> Rs 5000 PN")
    print("3. Economy class -> Rs 3000 PN")

    choice = int(input('Enter your choice: '))
    passengers = int(input('Number of passengers: '))
```

```
if choice == 1:
    price = 7000 * passengers
    class_name = "First class"
elif choice == 2:
    price = 5000 * passengers
    class_name = "Business class"
elif choice == 3:
    price = 3000 * passengers
    class_name = "Economy class"
else:
    print('Invalid choice!')
    return 0
```

```
print(f'You have opted {class_name}. Ticket fare = Rs
{price}\\n') with open("Bill.txt", 'a') as f:
    f.write(f"Ticket: {class_name}, Fare = {price}\\n")
return price
```

```
# Function to order food items
def orderitem():
    try:
        mycursor.execute("SELECT * FROM
        food") rows = mycursor.fetchall()
    except Error as e:
        print("Error fetching food items:", e)
        return 0
```

```
print("\nAvailable food items:")
```

```
        for row in rows:  
            print(f'{row[0]}. {row[1]} - Rs {row[2]}')
```

```
choice = input("Enter your choice (number) or 0  
to skip: ") if choice == '0':
```

```
    print("No food chosen.")  
    with open("Bill.txt", 'a') as f:  
        f.write("No food items chosen.\n")  
    return 0
```

```
quantity = int(input("Enter quantity: "))
```

```
item = next((item for item in rows if str(item[0]) ==  
choice), None) if item:
```

```
    total = item[2] * quantity  
    print(f'Your amount for {item[1]} = Rs {total}')  
    with open("Bill.txt", 'a') as f:  
        f.write(f'{item[1]} x {quantity} = Rs {total}\n')  
    return total
```

```
else:
```

```
    print("Invalid food choice.")  
    return 0
```

```
# Function to calculate luggage bill
```

```
def luggagebill():
```

```
    try:  
        mycursor.execute("SELECT * FROM luggage")  
        rows = mycursor.fetchall()  
    except Error as e:  
        print("Error fetching luggage info:", e)  
        return 0
```

```
print("\nAvailable luggage info (weight limits and  
charges):") for row in rows:
```

```
    print(f'{row[0]}. {row[1]} kg extra = Rs {row[2]} per kg')
```

```
choice = input("Do you have extra luggage? (y/n): ").lower()
```

```
if choice == 'y':
```

```
    weight = int(input("Enter weight of extra luggage (kg): "))
```

```

bill = weight * 1000
print(f"Luggage bill: Rs {bill}")
with open("Bill.txt", 'a') as f:
f.write(f"Luggage bill: Rs {bill}\n")
return bill
return 0

```

```

# Function to generate final bill
def generatebill():
print("\n----- Generating Bill -----\n")
try:
with open("Bill.txt", 'r') as f:
print(f.read())
except FileNotFoundError:
print("No bill found. Please register customer and choose services first.")

```

```

# FIXED Cancel Booking
def cancelbooking():
name = input("Enter the customer name to cancel booking: ")

```

```

# Check if customer exists
sql_check = "SELECT * FROM pdata WHERE custname = %s"
mycursor.execute(sql_check, (name,))
record = mycursor.fetchone()

```

```

# IMPORTANT FIX — clear unread rows
mycursor.fetchall()

```

```

if not record:
print("No such booking found!")
return

```

```

# Delete customer booking
sql_delete = "DELETE FROM pdata WHERE custname = %s"
try:
mycursor.execute(sql_delete, (name,))
con.commit()
print(f"Booking for {name} has been cancelled successfully.") with
open("Bill.txt", "a") as f:

```

```

f.write(f"Booking for {name} cancelled.\n")

```

```
except Error as e:  
    print("Error cancelling booking:", e)
```

```
# Main menu function
```

```
def Menuset():
```

```
    while True:
```

```
        print("\n===== Main Menu =====")  
        print("1: Enter customer data")  
        print("2: Services (Ticket, Food, Luggage)")  
        print("3: Generate bill")  
        print("4: Cancel Booking")  
        print("5: Exit")
```

```
    try:
```

```
        choice = int(input("Enter your choice: "))
```

```
    except ValueError:
```

```
        print("Enter a valid number.")  
        continue
```

```
    if choice == 1:
```

```
        registercust()
```

```
    elif choice == 2:
```

```
        ticket_total = ticketprice()  
        food_total = orderitem()  
        luggage_total = luggagebill()  
        total_amount = ticket_total + food_total +  
        luggage_total  
        print(f"\nTotal Amount to be paid  
= Rs {total_amount}")  
        with open("Bill.txt", 'a') as f:
```

```
            f.write(f"\nTotal Amount = Rs  
{total_amount}\n")  
    elif choice == 3:
```

```
        generatebill()
```

```
    elif choice == 4:
```

```
        cancelbooking()
```

```
    elif choice == 5:
```

```
        print("Exiting program. Goodbye!")  
        break
```

```
    else:
```

```
        print("Invalid choice. Try  
again.") # Start program
```

```
if __name__ == "__main__":
```

```
    Menuset()
```

ADVANTAGES AND LIMITATIONS

- **Advantages** • High performance
 - Flexibility
 - Data integrity
 - Portability
 - Data security
- **Disadvantages**
 - Make sure that the pip package is installed.
 - Check with mysql.connector if the connectivity is established between python and SQL.
 - Installation and adding paths to the system variables is a time taking process.



BIBLIOGRAPHY

○Python IDLE

(3.11) ○MySQL

[○www.google.com](http://www.google.com)

